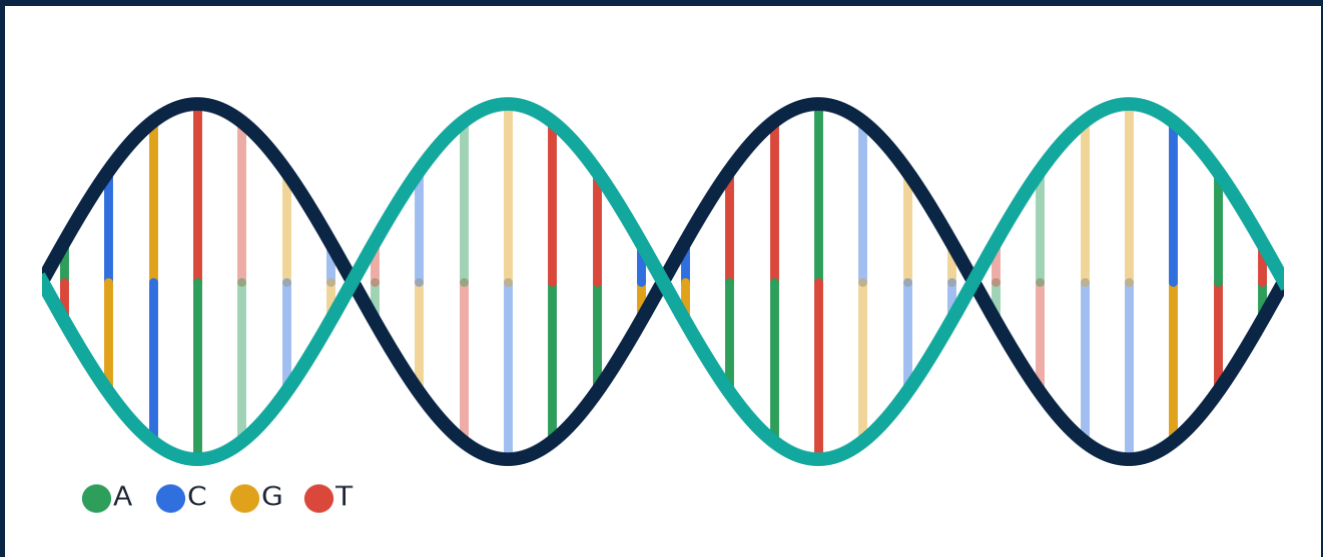


VNX-DNA

Computational DNA data storage, end to end



What it is · how it works · what has been proven · what investors and programmers ask

Version 2.0.0 · 30 September 2026

VS Nexus Technologies · [vishnusevram101-hash/VSnexustechnologies-](https://github.com/vishnusevram101)

Contents

1	VNX-DNA at a glance
2	Why store data in DNA?
3	How VNX-DNA works — the whole journey
4	Inside a DNA strand
5	Error correction: why nothing is lost
6	Reading the DNA back
7	Security and integrity
8	Proven results
9	Storage profiles
10	For investors
11	For programmers
12	Limitations — stated plainly
13	Glossary
A	Reproducing every number

How to read this document. Sections 1–3 explain VNX-DNA to anyone. Sections 4–9 are the technical core. Section 10 answers investor questions and Section 11 answers programmer questions. Every measured number in this document was generated by the released software on a real machine and is read directly from its result files; none is typed by hand.

1 VNX-DNA at a glance

VNX-DNA is software that turns any digital file into DNA sequences — strings of the four letters A, C, G and T — and turns those sequences back into the exact original file. It is the complete digital half of a DNA data storage system: the part that decides *what* DNA to make, and that later reconstructs the data from imperfect sequencing reads.

A DNA synthesis company can manufacture the sequences VNX-DNA writes. A sequencing machine can read them back. Everything in between — compression, encryption, error correction, sequence design, making sense of millions of noisy, shuffled, duplicated reads, and proving the result is exact — is what VNX-DNA does.

10 GB largest file stored and recovered exactly	0.60 GiB peak RAM at any size, 1 MB – 10 GB	100 % of 1,000 Monte Carlo trials recovered exactly	0 wrong files returned in 2,320 simulated trials
---	---	---	--

What is in version 2.0.0

- **Any file, any size.** Files are processed in small chunks, so memory use stays flat: a 10 GB file needed 0.58 GiB of RAM, about the same as a 1 GB file.
- **Three layers of error correction** — a checksum and Reed–Solomon code inside every strand, and a second Reed–Solomon code across strands — so lost and damaged strands are rebuilt with a mathematical guarantee.
- **Encryption built in** (AES-256-GCM). DNA made from an encrypted archive reveals nothing about the content.
- **A realistic simulated lab:** synthesis errors, uneven copy numbers, lost strands, sequencing errors, duplicates, reversed reads, junk and contamination — followed by clustering and consensus, just as real sequencing data needs.
- **Random access.** Any byte range of a huge archive can be recovered without decoding the rest.
- **Proof of exactness.** Output is released only when its SHA-256 fingerprint matches the original. When recovery is impossible, VNX-DNA says so, names the damaged chunk and writes nothing.
- **406 automated tests**, all passing on a clean install; open, reproducible benchmarks.

Honest scope. VNX-DNA is software. Every result in this document comes from computation and simulation. No DNA has yet been synthesised or sequenced with VNX-DNA. The next milestone is a wet-lab pilot (Section 10).

2 Why store data in DNA?

The world creates data faster than it can afford to keep it. IDC forecast the global datasphere to reach 175 zettabytes by 2025 [1]. Most of that data is rarely read but must be kept for years or decades — medical images, legal and financial records, scientific datasets, film and cultural archives. Today it sits on magnetic tape and hard disks, which must be powered, cooled, and migrated to new media every few years.

DNA is nature's storage medium and it has three properties no electronic medium matches:

- **Density.** DNA stores information at the molecular scale. Published work has estimated that DNA can hold on the order of 215 petabytes per gram [2] — a data-centre's worth of data in a volume smaller than a sugar cube.
- **Longevity.** Kept cool and dry (for example encapsulated in silica) DNA has been estimated to remain readable for thousands of years [3], without power and without migration.
- **It never becomes obsolete.** As long as life sciences exist, people will be able to read DNA. Tape formats and drive interfaces change every few years; the genetic code does not.

Researchers have demonstrated the idea repeatedly: a book stored in DNA in 2012 [4], 739 kB in 2013 [5], the 'DNA Fountain' codec in 2017 [2], and 200 MB with random access in 2018 [6].

What stands in the way

Two things. First, **cost and speed of DNA synthesis** — the chemistry industry is working on this. Second, **the data problem**: synthesis and sequencing are error-prone. Strands are lost, letters are swapped, added or dropped, reads come back in random order and in wildly uneven numbers. Turning that mess back into a perfect file, at scale, with a guarantee, is a hard software and coding-theory problem. **That is the problem VNX-DNA solves.**

[1] Reinsel, Gantz, Rydning, *Data Age 2025*, IDC/Seagate, 2018. [2] Erlich & Zielinski, *DNA Fountain enables a robust and efficient storage architecture*, Science 355, 2017. [3] Grass et al., *Robust chemical preservation of digital information on DNA in silica with error-correcting codes*, Angew. Chem. 54, 2015. [4] Church, Gao, Kosuri, *Next-generation digital information storage in DNA*, Science 337, 2012. [5] Goldman et al., *Towards practical, high-capacity, low-maintenance information storage in synthesized DNA*, Nature 494, 2013. [6] Organick et al., *Random access in large-scale DNA data storage*, Nature Biotechnology 36, 2018.

3 How VNX-DNA works — the whole journey

VNX-DNA is a chain of steps. Each step is a separate command that writes a file you can inspect and verify, so the chain can be run one step at a time or all at once with `vnx-dna pipeline`.

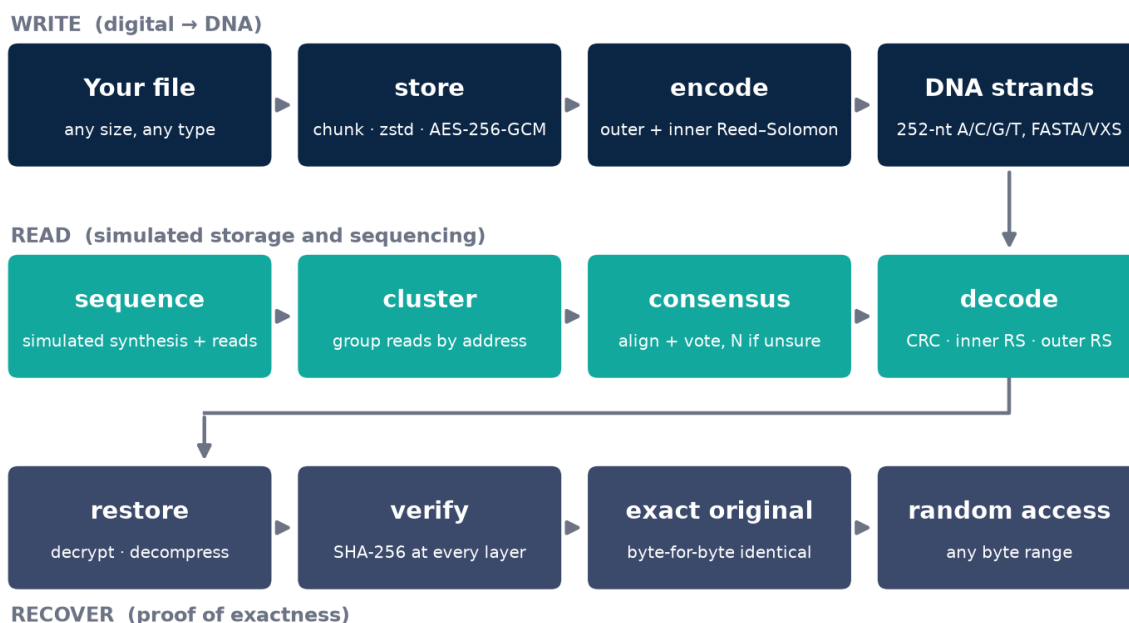


Figure 1 — The VNX-DNA pipeline. The top row writes DNA, the middle row reads it back through a simulated lab, the bottom row proves the result is exact.

Step by step

Step	What happens
1. store	The file is cut into fixed-size chunks (1 MiB by default). Each chunk is compressed with <code>zstd</code> (only if that makes it smaller) and, if a key is given, encrypted with AES-256-GCM. Chunks go into a <code>.vxdna</code> container with an index and a SHA-256 fingerprint of everything. Memory never holds more than a few chunks.
2. encode	Each stored chunk is split into groups of 64 pieces of 40 bytes. For every group, 16 extra <i>parity</i> pieces are computed (outer Reed-Solomon code). Every piece gets an address header, a CRC checksum and 8 bytes of inner Reed-Solomon parity, and is written as 252 DNA letters (2 bits per letter).
3. sequence rules	Real DNA chemistry dislikes long runs of one letter (AAAAA) and too much or too little G+C. VNX-DNA scrambles each strand with one of 256 reversible patterns until it satisfies the rules, and records which pattern it used inside the strand. If no pattern works, it fails loudly rather than emitting bad DNA.
4. synthesise and store	In a real deployment a synthesis provider manufactures the strands. In VNX-DNA today this, and sequencing, are simulated : strands are lost, copied unevenly, and read with substitution, insertion and deletion errors, reversed, shuffled, duplicated and mixed with junk.
5. cluster and consensus	Reads of the same strand are grouped using the address written inside the DNA itself. Each group is aligned and a quality-weighted vote produces one best-guess strand. Where the vote is unclear the base is marked N (unknown) instead of guessed — the error-correction code handles a known unknown twice as efficiently as a wrong guess.
6. decode	Each strand is checked (CRC), repaired if needed (inner RS), and sorted into its group. Missing strands are rebuilt by the outer code. The container is rebuilt byte for byte.
7. restore and verify	Chunks are authenticated, decrypted, decompressed and fingerprinted. The output file appears only if the whole-file SHA-256 matches the original.

4 Inside a DNA strand

Every strand is self-describing. Its address — which archive, which error-correction group, which position in the group — is written in the DNA itself, not in a file name or a FASTA header. A test tube of mixed, shuffled strands therefore contains everything needed to reassemble the archive.

One strand = 63 bytes → 252 nucleotides (2 bits per base)

balanced profile · the header travels inside the DNA, so file names or FASTA headers are never needed



G T T A A A T G G C A G A A A A C T G G C A G G G C T T T T A G T C G T G G G A T G A T C A G T G G G T A A A G G T G G C G C

illustrative strand sequence (63 of its 252 nt shown); real strands are screened for GC 40–60 % and runs ≤ 4

Figure 2 — Layout of one strand in the balanced profile. 40 of 63 bytes carry data; the rest make the strand findable, checkable and repairable.

Density

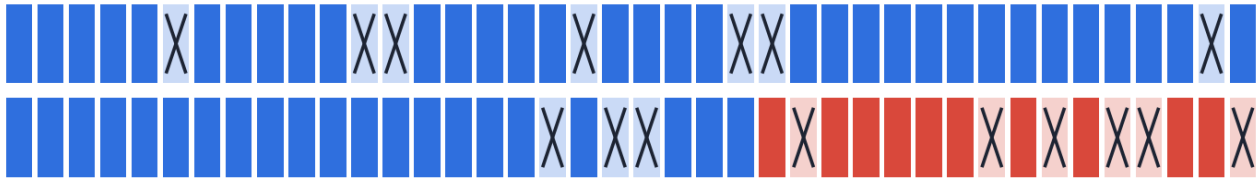
For the 10 GB test file, VNX-DNA produced 138,563,183 strands and 34,917,922,116 bases — **3.49 bases per input byte** after compression, all error correction, addressing and metadata. That is 2.29 bits of original data per base, against a theoretical maximum of 2 bits per base for uncompressible data. (The test file is a 'mixed' pattern that is partly compressible; profile trade-offs are in Section 9.)

Field	Size	Purpose
scrambler variant	1 byte	which of 256 patterns made the strand satisfy the DNA rules
format + kind	1 byte	frame version 5; data/parity or metadata strand
archive tag	4 bytes	which archive the strand belongs to (strands from different archives can be mixed)
ECC group	4 bytes	which error-correction group (up to 4.3 billion groups, ~11 TB of stored data)
shard	1 byte	position inside the group (data first, then parity)
payload	40 bytes	the (compressed, encrypted) data itself
CRC-32	4 bytes	detects a damaged strand
inner RS parity	8 bytes	repairs up to 4 damaged bytes inside the strand

5 Error correction: why nothing is lost

DNA storage loses whole strands (they are never synthesised, never sampled, or never read) and corrupts letters inside strands. VNX-DNA fights each with a dedicated code, and verifies the result with cryptographic fingerprints.

One ECC group: 64 data strands (blue) + 16 parity strands (red)



Any 16 of the 80 strands may be lost (X) and the group is still rebuilt exactly — Cauchy Reed-Solomon is MDS.

Lose 17 in one group and VNX-DNA stops with exit code 5, names the chunk, and writes nothing.

Figure 3 — The outer code. Every group of 64 data strands gets 16 parity strands; any 16 of the 80 may disappear.

The outer code is a **Cauchy Reed-Solomon** code. It is *MDS* (maximum distance separable) — the best any code can do: with 16 parity strands, *any* 16 losses are recoverable, not just most patterns. This was proven and verified exhaustively on small codes. (An early prototype, V0.1, used a code that was not MDS; VNX-DNA 1.0 found and replaced it.)

1 CRC-32 on every strand

throws away a damaged read instead of trusting it

2 Inner Reed-Solomon (8 parity bytes)

fixes up to 4 wrong bytes in a strand, or 8 at known-bad positions

3 Outer Cauchy Reed-Solomon (64 + 16)

rebuilds any 16 missing strands out of every 80

4 AES-256-GCM tag + chunk SHA-256

proves every chunk is authentic, complete and in the right place

5 Whole-file SHA-256 + atomic rename

the output file appears only if it matches the original exactly

Figure 4 — Five independent layers stand between a damaged test tube and a wrong file.

The guarantee in one sentence. If no error-correction group loses more than 16 of its 80 strands, the file is recovered exactly; if one does, VNX-DNA exits with code 5, names the chunk, and writes nothing. It never returns a corrupted file as if it were correct.

6 Reading the DNA back

A sequencer does not return 'the strands'. It returns millions of short reads: each strand appears many times (coverage), some not at all, in random order, some reversed, each with its own errors. VNX-DNA handles this in four stages, all streaming and disk-backed so they scale:

Stage	What it does	Why it matters
read filtering	validates reads, drops junk, applies length and quality limits	garbage never reaches the decoder
clustering	groups reads by the address written in the DNA; a minimizer index catches reads whose address is damaged	most reads cluster in constant time — no all-against-all comparison
consensus	aligns each cluster (banded alignment) and takes a quality-weighted vote; unclear bases become N	turns 10 noisy copies into one clean strand and handles insertions/deletions
two-pass decoding	pass 1 checks and repairs every read and spills it to disk; pass 2 sorts by group and runs the outer code	memory is independent of the number of reads or their order

Insertions and deletions (letters added or dropped) are the hardest errors in DNA storage because they shift everything after them. Reed–Solomon codes cannot fix shifts, and VNX-DNA does not pretend they can: consensus alignment across multiple reads re-synchronises the strand first, and an optional single-read realignment handles one indel per read. Section 8 shows how far this goes.

Simulated lab: what the channel model includes

- strand dropout and uneven copy numbers (Poisson or log-normal abundance)
- synthesis and sequencing substitutions, insertions and deletions; truncated reads; N calls
- duplicates, reverse-complement reads, junk reads and contamination from other archives
- informative quality scores, full shuffling of reads, FASTQ/FASTA/VXS output
- every event is counted as it actually happened, and every run is reproducible from its seed

The channel is a **stress model**, not a model fitted to a specific sequencing platform. Fitting it to real wet-lab data is part of the pilot (Section 10).

7 Security and integrity

- **Encryption.** AES-256-GCM per chunk with keys derived by HKDF-SHA256. File name, size and hashes are sealed inside the encrypted manifest, which is authenticated with HMAC-SHA256. Encoding and decoding DNA need no key; only restoring the plaintext does.
- **Tamper evidence.** Each chunk's authentication is bound to its position and to the total chunk count, so modified, reordered, duplicated, missing, truncated or spliced chunks are all detected. Format version labels in the key derivation prevent downgrade attacks.
- **Crash safety.** Every output is written to a temporary file, flushed to disk and renamed into place atomically. A killed process can never leave behind a file that looks valid. An interrupted store resumes with `--resume` and produces a byte-identical archive (tested with SIGKILL).
- **No silent failure.** Errors are one line with a stable exit code (0-8, 70). A bug shows as exit code 70, never as a wrong file.
- **Code hygiene at release.** No secrets or keys in the repository, no eval/exec/pickle or shell execution, no debug leftovers; fuzzed with thousands of malformed containers and manifests.

What encryption does not hide: the approximate size of the archive and how compressible each chunk was. This is documented in docs/SECURITY.md.

8 Proven results

All measurements below were produced by the released code (v2.0.0rc4, commit 9f25598, clean tree) on a 8-CPU x86-64 Linux machine with Python 3.12.3, CPU only, no GPU.

8.1 Large files: 1 MB to 10 GB

Each file was generated, stored, encoded to DNA, recovered from the DNA, restored, and compared with the original using both SHA-256 and a byte-by-byte comparison. A 1 MiB region near the middle was also extracted directly from the DNA file (random access) and compared with the original bytes.

Input	Strands	DNA bases	Store	Encode	Recover	Peak RAM	Result
1 MB	14,282	3,599,064	0.4 s	0.6 s	0.6 s	0.32 GiB	exact ✓
10 MB	138,979	35,022,708	0.5 s	0.8 s	0.8 s	0.39 GiB	exact ✓
100 MB	1,386,073	349,290,396	1.0 s	2.2 s	2.6 s	0.54 GiB	exact ✓
500 MB	6,927,809	1,745,807,868	3.3 s	8.4 s	10.0 s	0.53 GiB	exact ✓
1 GB	13,855,866	3,491,678,232	6.2 s	16.4 s	21.4 s	0.60 GiB	exact ✓
2 GB	27,712,598	6,983,574,696	12.8 s	33.0 s	40.9 s	0.56 GiB	exact ✓
5 GB	69,281,261	17,458,877,772	31.0 s	81.9 s	1.7 min	0.57 GiB	exact ✓
10 GB	138,563,183	34,917,922,116	61.4 s	2.7 min	3.3 min	0.58 GiB	exact ✓

Table 1 — Scale matrix. 'Exact' means SHA-256 and byte comparison matched for the whole file and for the random-access region. Peak RAM is the whole process tree.

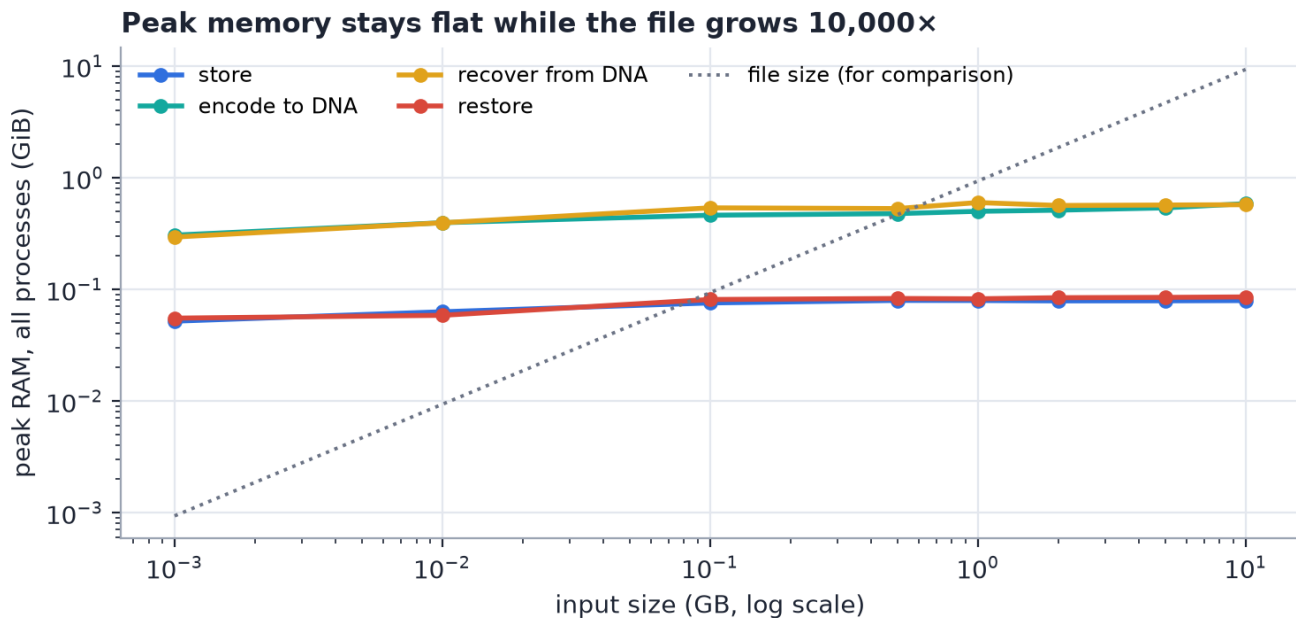


Figure 5 — The key scalability result: memory depends on chunk size and worker count, not on file size.

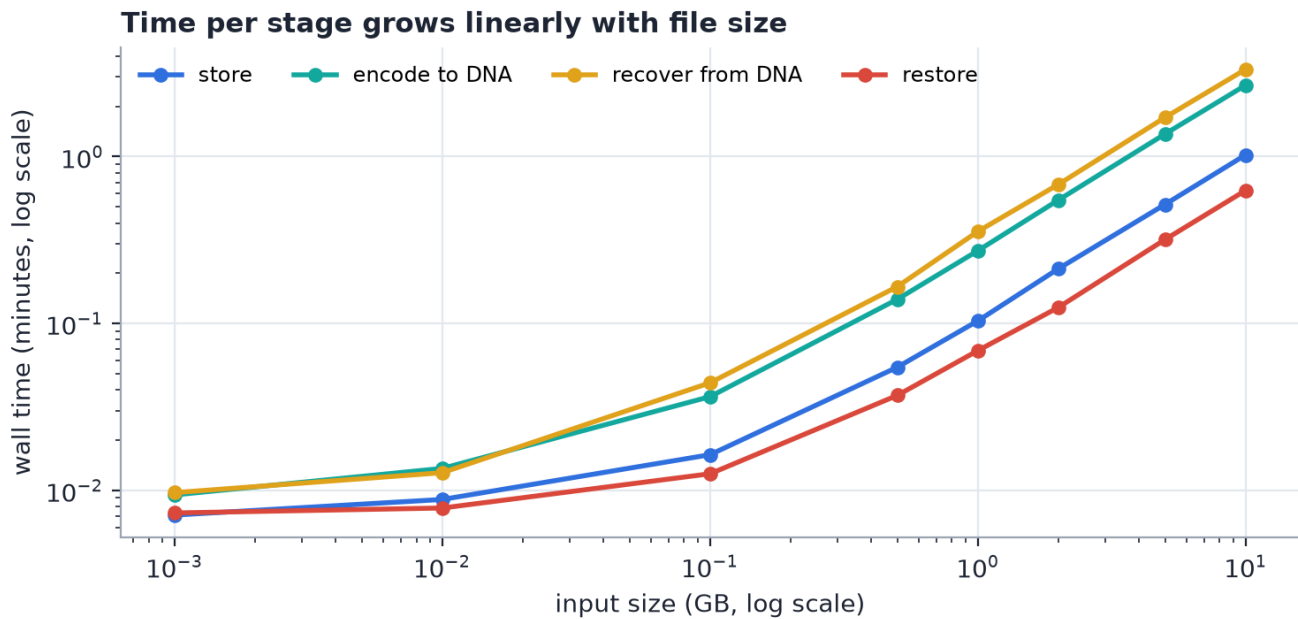


Figure 6 — Time grows linearly with size. End-to-end store + encode + recover averaged 24 MB/s at 10 GB on 8 CPUs. Peak scratch disk for 10 GB was 36 GB.

8.2 Damage at scale (1 GB)

On the real strand file of a 1 GB archive, 800 strands were deleted and 250 more were corrupted, spread over 50 error-correction groups — each group losing exactly 16 strands, the limit of the guarantee. Recovery was **exact (SHA-256 and byte comparison matched)**. A second run repeated that damage and added one more group that lost 17 strands — one beyond the guarantee: VNX-DNA exited with code 5, wrote no output, and reported:

```
vnx-dna: error [INSUFFICIENT_REDUNDANCY]: chunk 590: 1 ECC group(s) lost more than 16 strands (worst lost 17); the outer code guarantees recovery of at most 16 per group
```

8.3 The simulated lab: how much damage can it take?

These experiments push a 20 kB file through the full chain — simulated synthesis, storage, sequencing, clustering, consensus, decoding — many times with different random seeds, and count how often the exact file comes back. Shaded bands are 95 % confidence intervals.

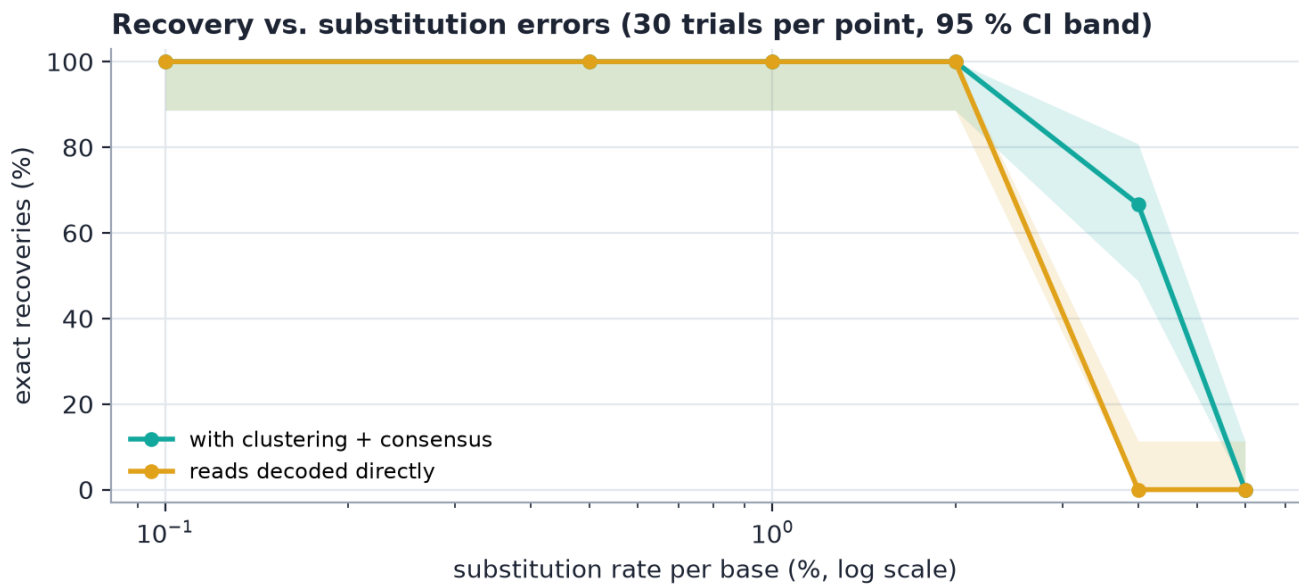


Figure 7 — Letter substitutions. Recovery is perfect up to 2 % of bases wrong; consensus extends the range further. At 6 % the reads are so damaged that no strand can be identified, and VNX-DNA reports that.

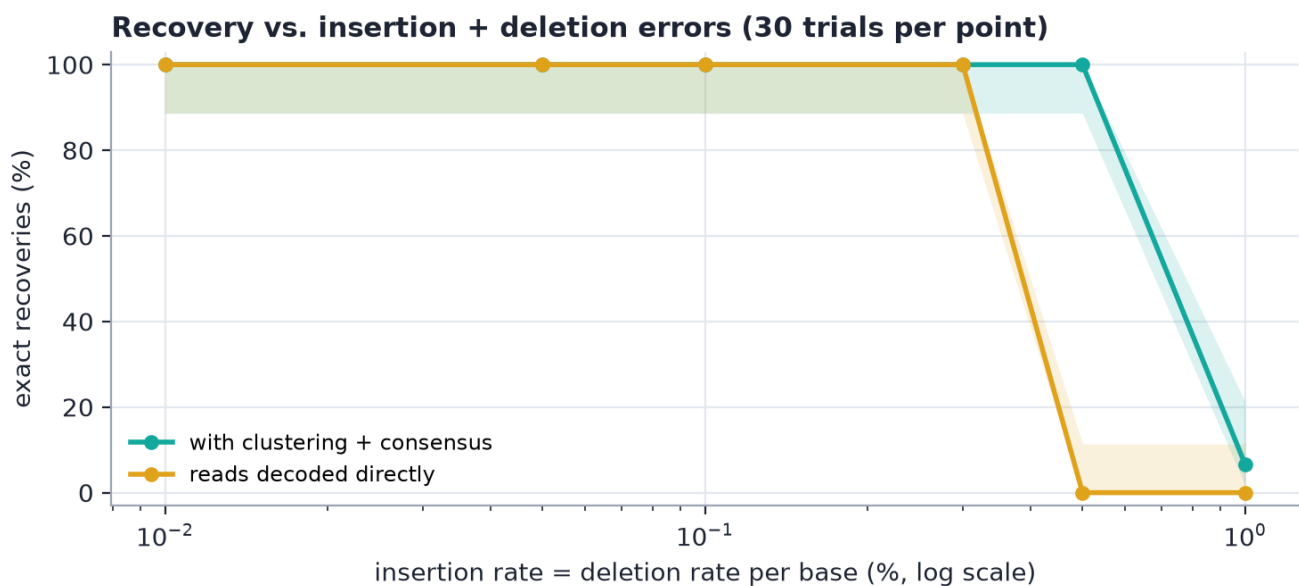


Figure 8 — Insertions + deletions (each at the given rate). Reads decoded directly cope up to 0.3 % per base; with consensus, up to 0.5 %. Beyond that recovery fails — cleanly, never with a wrong file.

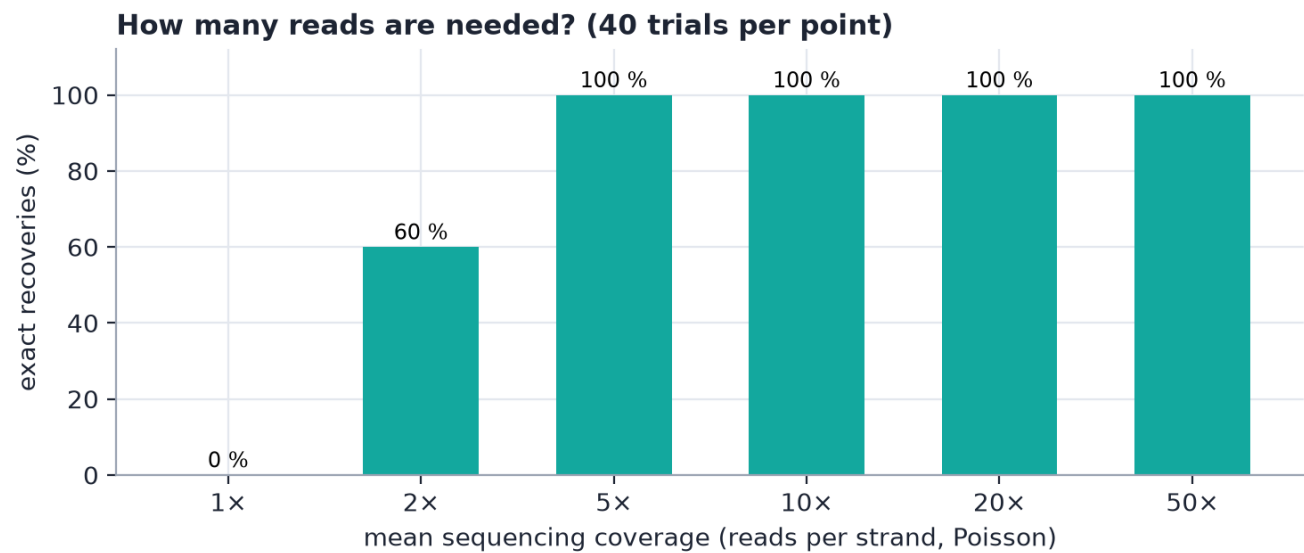


Figure 9 — Reads per strand. At 1× coverage too many strands are never read; from 5× upwards recovery was 100 %.

Monte Carlo, canonical channel. 1,000 independent trials at 10× coverage with 2 % dropout, 0.1 % substitutions and 0.01 % insertions and deletions: **1,000 exact recoveries** (95 % lower confidence bound 99.6 %). Across all 2,320 simulated trials in this section, the number of wrong files returned was **0** and the number of internal errors was **0**.

9 Storage profiles

One size does not fit all. More redundancy costs more DNA (and synthesis money) but survives worse conditions. VNX-DNA ships four named profiles; every parameter can also be set individually.

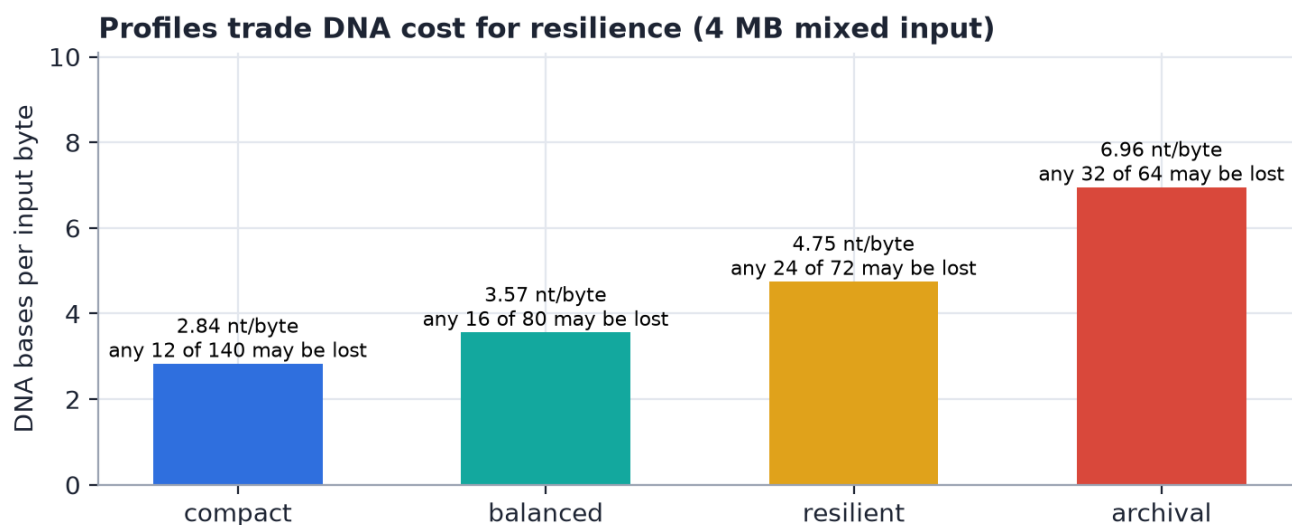


Figure 10 — Measured DNA cost per input byte on a 4 MB mixed file.

Profile	Bases / byte	Strand	Outer code guarantee	Inner repair	Harsh-channel success
compact	2.84	276 nt	any 12 of 140 strands lost	3 bytes / strand	0 %
balanced	3.57	252 nt	any 16 of 80 strands lost	4 bytes / strand	59 %
resilient	4.75	252 nt	any 24 of 72 strands lost	6 bytes / strand	100 %
archival	6.96	252 nt	any 32 of 64 strands lost	8 bytes / strand	99 %

Table 2 — 'Harsh-channel success' is the share of 100 simulated trials recovered exactly under a deliberately severe channel (the same channel for all four profiles).

compact minimises synthesis cost for well-controlled conditions; **balanced** is the default; **resilient** survives far worse loss; **archival** doubles the data with parity and uses stricter sequence rules for the longest storage.

10 For investors

The opportunity

Long-term 'cold' data is growing faster than the budgets that pay to store it. DNA offers a medium that is dense, durable for millennia and immune to format obsolescence. The chemistry — making and reading DNA — is being industrialised by synthesis and sequencing companies. What every DNA storage system also needs is the **codec**: robust, scalable, verifiable software that turns files into manufacturable DNA and imperfect reads back into perfect files. VNX-DNA is that layer, and it is independent of which chemistry wins.

Where VNX-DNA is today



What has been proven — and what has not

Proven (software, measured)	Not yet proven
Exact store → DNA → recovery of files up to 10 GB with flat memory (≈ 0.6 GiB)	Any result with physically synthesised and sequenced DNA
Mathematically guaranteed recovery of lost strands (MDS outer code)	Channel model fitted to a real sequencing platform
Recovery from simulated reads with substitutions, indels, dropout, duplicates, contamination	Cost per gigabyte in real synthesis; synthesis-vendor sequence rules
Zero wrong files in every simulated trial; clean failure beyond the guarantee	Molecular (PCR-based) random access; primer design; secondary structure
Encryption, tamper detection, crash safety; 406 automated tests	Long-term physical storage

Investor questions

Is this a real product or a research demo?

It is working, tested, installable software (v2.0.0) that runs the entire digital side of DNA storage on an ordinary CPU server. It is not yet a commercial service: DNA has not been physically made with it. The honest description is 'production-quality codec, pre-wet-lab'.

What is the next milestone, and what does it prove?

A wet-lab pilot: synthesise a pool of VNX-DNA strands through a commercial provider, sequence it, and recover the file. It converts simulated evidence into physical evidence and supplies real error data to fit the channel model.

What is defensible here?

The engineering of the whole chain at scale: a strand format with in-DNA addressing, a provably optimal outer code with inner repair, consensus that marks uncertainty instead of guessing, streaming decoders with flat memory, and a verification discipline that has never returned a wrong file in testing. Each piece exists in the literature; making them work together at 10 GB scale, reproducibly, is the hard part.

How could this make money?

Options include licensing the codec/SDK to DNA synthesis and storage companies, offering an archive service with a synthesis partner, and professional services for institutions that must keep data for decades. Which model is pursued is a business decision beyond the scope of this technical document.

What are the main risks?

(1) Synthesis cost: DNA storage becomes mainstream only as synthesis gets cheaper — outside VNX-DNA's control. (2) Real channels may differ from the simulation; the pilot addresses this, and profiles already trade cost for robustness. (3) Throughput: today's CPU implementation processes roughly 24 MB/s end to end on 8 CPUs; faster decoders are on the roadmap. (4) Competition from other codecs; VNX-DNA's answer is scale, verification and openness.

How much DNA does a gigabyte need?

About 3.5 billion bases per GB in the default profile on our mixed test data (3.49 bases per byte, all overheads included); fewer with the compact profile or more compressible data.

Can results be checked independently?

Yes. Every number here is produced by commands in the public repository and stored as JSON with the exact commit and environment. Appendix A lists the commands.

11 For programmers

Install

Linux, Python ≥ 3.12 , CPU only. Dependencies: numpy, cryptography, zstandard, reedsolo, pydantic, typer.

```
git clone https://github.com/vishnuseelvam101-hash/VSnexustechnologies-.git
cd VSnexustechnologies-
git checkout v2.0.0
python3 -m venv .venv && . .venv/bin/activate
pip install -e '[dev]'          # '.' without dev extras to skip the test tools
vnx-dna --help
python -m pytest                # 406 tests
```

Hello, DNA

```
echo "Hello VNX-DNA" > input.txt
vnx-dna store input.txt --output hello.vxdna      # streaming container (format 5)
vnx-dna encode hello.vxdna --output hello.fasta   # DNA strands + hello.fasta.vxidx
vnx-dna recover hello.fasta --output recovered.txt # DNA -> verified original
cmp input.txt recovered.txt && echo identical
```

The full chain, one command per stage

```
vnx-dna benchmark generate --size 200KB --pattern mixed --seed 42 --output input.bin
vnx-dna store input.bin --output archive.vxdna [--key-file key.txt] [--profile resilient]
vnx-dna encode archive.vxdna --output strands.fasta
vnx-dna sequence strands.fasta --coverage 10 --substitution-rate 0.001 \
    --insertion-rate 0.0001 --deletion-rate 0.0001 --dropout-rate 0.02 --seed 42 --output reads.fastq
vnx-dna cluster reads.fastq --output clusters.jsonl
vnx-dna consensus clusters.jsonl --output consensus.fasta
vnx-dna decode consensus.fasta --output recovered.vxdna
vnx-dna restore recovered.vxdna --output recovered.bin
vnx-dna verify recovered.vxdna --file recovered.bin
# or everything at once, with a JSON report:
vnx-dna pipeline input.bin --output out.bin --coverage 10 --seed 42 --report pipeline.json
```

Large files and random access

```
vnx-dna store big.bin --output big.vxdna --resume          # resumable
vnx-dna encode big.vxdna --output big.vxs                 # packed 2 bits/base + DNA index
vnx-dna recover big.vxs --output big.out --temp-dir /scratch
vnx-dna extract big.vxs --offset 500000000 --length 1048576 --output part.bin
vnx-dna benchmark scale --sizes 1GB,10GB --work-dir /scratch --output scale.json
```

Python API (streaming)

```
from vnxdna.v2 import api
api.store("big.bin", "big.vxdna", workers=8)      # resume=True to continue
api.encode("big.vxdna", "big.vxs")
api.recover("big.vxs", "big.out")                 # two-pass, disk-backed decoder

from vnxdna.v2.strandio import iter_batches
for batch in iter_batches("reads.fastq", batch_reads=65536):
    batch.codes, batch.lengths, batchquals        # flat numpy arrays
```

Code map

Module	Responsibility
vnxdna.cli	Typical CLI (thin); vnxdna v1 ... mounts the unchanged V1 CLI
vnxdna.v2.api	public API; detects V1/V2 inputs and dispatches
v2.archive / v2.container	streaming store, restore, verify, extract; .vxdna v2 writer/reader
v2.crypto / v2.manifest	HKDF, chunked AES-256-GCM, HMAC; manifest, chunk and plaintext indexes
v2.frame / v2.crc / v2.constraints	strand frame 5, vectorised CRC-32, inner RS, sequence-rule screening
v2.encoder / v2.decoder	chunk-parallel encoding and DNA index; two-pass disk-backed decoding
v2.sequencing	simulated synthesis/storage/sequencing channel
v2.cluster / consensus / align / sync	read clustering, consensus, banded alignment, realignment
v2.experiment / scale / stages	Monte Carlo experiments, test data, resource-measured benchmarks
vnxdna.* (top level)	V1 (format 4) modules, unchanged, for reading old archives

Exit codes

Code	Meaning
0	success (also when repair was needed: status RECOVERED)
1	verification failed
2	usage error
3	invalid input (missing, truncated, malformed)
4	authentication failed / wrong key
5	insufficient redundancy — too much damage; nothing written
6	unsupported format or feature
7	configuration error (e.g. unsatisfiable sequence rules)
8	output error (exists, not writable)
70	internal error (a bug; VNXDNA_DEBUG=1 for a traceback)

Programmer questions

Does it load the whole file into memory?

No. Every stage processes bounded blocks: at most about $2 \times$ workers chunks are in flight. What grows with the file is disk (output, decoder spill, shuffle buckets) — point `--temp-dir` at a big volume.

Is output deterministic?

Yes for unencrypted archives (content-derived archive ID) and for every simulation (seeded). Experiments give identical results regardless of worker count (tested).

Can old archives still be read?

Yes. V1 (format 4) containers and reads are read by every command using the exact V1 code; `vnxdna migrate` converts them with verification before and after.

How do I add a sequence rule or a mapping?

Constraints live in `vnxdna.v2.constraints` and are part of the stored options, so decoders know what was used. Mappings (2bit, rotation3, codebook8) are selected per archive. Add tests in `tests/v2` that prove round-trips and that unsatisfiable settings fail with exit code 7.

What is tested?

406 tests: unit, integration, property-based (Hypothesis), adversarial/fuzz, CLI subprocess tests including the README examples, bounded-memory tests, SIGKILL-and-resume tests, exhaustive ECC boundary tests (exactly M losses recover, M + 1 fail).

Where are the formats specified?

`docs/V2_FORMAT.md` (container, manifest, indexes, frame 5, VXS, DNA index).

`docs/V2_ARCHITECTURE.md` lists every design decision and where each guarantee is tested.

12 Limitations — stated plainly

- **Software only.** No wet-lab synthesis or sequencing yet. The channel is a stress model, not a platform model. Sequence rules are common heuristics, not a synthesis vendor's specification; secondary structure and primer design are not modelled.
- **Guarantees are per error-correction group.** Any 16 of 80 strands per group may be lost (balanced profile). Beyond that, success under random damage is a measured probability, not a promise.
- **Indels** need coverage above 1× (consensus) or the opt-in single-read realignment (one indel per read). Strands have no in-strand synchronisation markers.
- **Scale of the sequencing simulation.** Store, encode, decode, restore and random access are tested at 10 GB. The full simulated sequencing → clustering → consensus chain is measured on inputs up to 10 MB, because 10× coverage of 10 GB means roughly 6×10^{11} sequenced bases (about 1.2 TB of reads). Memory is not the limit; time and disk are.
- **Throughput** is CPU-bound Python/numpy, about 24 MB/s end to end on 8 CPUs; inner-RS decoding and consensus alignment are the slowest stages for damaged reads.
- **Encrypted archives reveal** their approximate size and per-chunk compressibility.
- **Random access** reads only the needed records of a strand *file*; it is not molecular (PCR-based) random access.

13 Glossary

Term	Meaning
nucleotide / base	one DNA letter: A, C, G or T; VNX-DNA stores 2 bits per base before overheads
strand / oligo	a short synthetic DNA molecule; here 252 bases
synthesis	manufacturing DNA strands with a chosen sequence
sequencing / read	reading DNA; each read is one noisy observation of one strand
coverage	average number of reads per strand
dropout	a strand that is never read
substitution / insertion / deletion	a wrong, extra or missing letter in a read; insertions and deletions are 'indels'
cluster / consensus	grouping reads of the same strand / combining them into one best-guess strand
Reed-Solomon (RS)	an error-correcting code; 'MDS' means it recovers the maximum possible number of losses
CRC-32	a 4-byte checksum that detects damage
SHA-256	a cryptographic fingerprint; any change produces a different value
AES-256-GCM	authenticated encryption: secrecy plus tamper detection
GC content / homopolymer	share of G and C letters / a run of one repeated letter; both constrained for chemistry
FASTA / FASTQ / VXS	text strand formats (FASTQ adds quality scores) / VNX-DNA's packed 2-bit strand file

A Reproducing every number

Release: **v2.0.0**, commit b6c2c5b, repository

github.com/vishnuseelvam101-hash/VSnexusustechologies-. Measurements were taken on the release-candidate commit 9f25598 (v2.0.0rc4); the release commit differs only in version number and documentation.

```
python -m venv /tmp/rel && /tmp/rel/bin/pip install -e '[dev]'
```

```
python -m pytest
```

```
vnxdna benchmark scale --sizes 1MB,10MB,100MB,500MB,1GB,2GB,5GB,10GB \
```

```
    --work-dir /scratch --output research/results/v2/scale-matrix.json
```

```
vnxdna benchmark corruption --size 1GB --work-dir /scratch --output research/results/v2/corruption.json
```

```
python research/v2/run_v2_research.py --out research/results/v2      # all channel experiments (~1 h)
```

```
python research/v2/render_v2_tables.py                             # regenerates the tables in docs/
```

Environment of the measurements

Item	Value
VNX-DNA	2.0.0rc4
commit	9f255985304c78e267c0fb8027cb3240d444095b
platform	Linux-6.8.0-139-generic-x86_64-with-glibc2.39
CPUs	8
Python	CPython 3.12.3
libraries	cryptography 46.0.7, numpy 2.5.3, zstandard 0.25.0

Result files: research/results/v2/*.json (scale-matrix, corruption, errors, coverage, abundance, profiles, chunks, stages, montecarlo). Each records its inputs, seeds, environment and commit.